TRƯỜNG ĐẠI HỌC FPT

BÁO CÁO THỰC HÀNH

LAB-02: EMBEDDED LINUX SYSTEMS

Họ và tên	Lê Thế Trường
Mã số sinh viên	SE202014
Nhóm	1
Ngày thực hiện	30/10/2026
Môi trường	Ubuntu Host, QEMU ARM Versatile Express

TP. Hồ Chí Minh – 2026

MỤC LỤC

- 1. Mục tiêu và phạm vi thực hiện
- 2. Môi trường thực nghiệm
- 3. Thiết kế và kiến trúc Character Device Driver
- 4. Kết quả thực nghiệm
- 5. So sánh các hệ thống tập tin nhúng
- 6. Câu hỏi phân tích
- 7. Khó khăn và cách khắc phục
- 8. Kết luận

Ghi chú: Báo cáo được tổng hợp từ kết quả lệnh và log thực nghiệm đã ghi nhận. Phần JFFS2/NAND Simulator được chạy trên Ubuntu Host; phần Character Driver, Procfs/Sysfs và khởi động Initramfs được kiểm tra trong QEMU ARM.

1. MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

Bài thực hành LAB-02 nhằm tìm hiểu cách xây dựng và tích hợp một Character Device Driver trong Linux, trao đổi dữ liệu giữa user space và kernel space, cung cấp thông tin trạng thái thông qua Procfs/Sysfs, thử nghiệm filesystem JFFS2 trên NAND flash giả lập và tự động nạp driver trong quá trình khởi động hệ thống nhúng.

Các nội dung đã thực hiện gồm: (A) xây dựng và kiểm tra driver ký tự; (B) xuất thông tin qua Procfs và Sysfs; (C) tạo, ghi, mount và kiểm tra JFFS2 trên NAND Simulator; (D) cấu hình BusyBox Init để tự động nạp module khi khởi động.

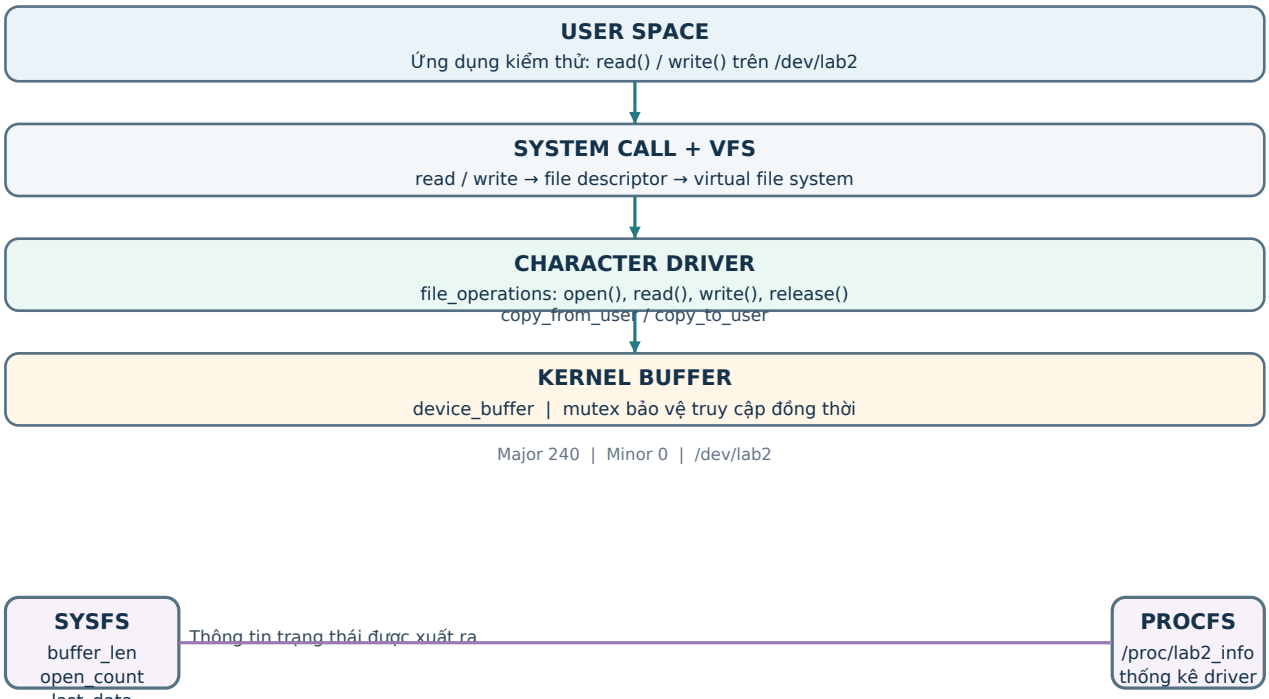
2. MÔI TRƯỜNG THỰC NGHIỆM

Thành phần	Thông tin
Máy chủ thực nghiệm	Ubuntu 24.04 (theo phiên bản GCC/toolchain được ghi nhận)
Host Kernel	7.0.0-38-generic
GCC Host	13.3.0
ARM Cross Compiler	arm-linux-gnueabihf-gcc 13.3.0
QEMU	8.2.2
Kernel mục tiêu	Linux 5.15.0, ARMv7, Cortex-A9
Machine QEMU	vexpress-a9
RAM QEMU	512 MB; 2 CPU cores
Root filesystem	BusyBox-based Initramfs, nén gzip
Cấu hình Kernel	CONFIG_ARM=y; CONFIG_ARCH_VEXPRESS=y; CONFIG_CPU_V7=y; CONFIG_MTD=y; CONFIG_JFFS2_FS=y

Kernel 7.0.0-38-generic là kernel của Ubuntu Host. Kernel 5.15.0 là kernel mục tiêu được khởi chạy trong QEMU. Hai phiên bản có vai trò khác nhau và không được nhầm lẫn khi mô tả kết quả.

3. THIẾT KẾ VÀ KIẾN TRÚC CHARACTER DEVICE DRIVER

Driver có tên lab2_driver, đăng ký Character Device với major number 240. Thiết bị được tạo ở user space dưới tên /dev/lab2 (minor 0). Cấu trúc file_operations cung cấp các callback để xử lý thao tác mở, đọc, ghi và đóng thiết bị. Dữ liệu trao đổi được lưu trong device_buffer; mutex lab2_mutex bảo vệ vùng dữ liệu dùng chung.



Luồng ghi dữ liệu: ứng dụng gọi write() → system call/VFS → callback write của driver → copy_from_user() sao chép dữ liệu hợp lệ từ user space vào device_buffer. Luồng đọc diễn ra theo chiều ngược lại, driver dùng copy_to_user() để đưa dữ liệu từ kernel buffer về user space.

Các giao diện quan sát trạng thái:

• Procfs: /proc/lab2_info cung cấp thông tin tổng hợp như tên driver, major number, độ dài buffer, số lần mở thiết bị và tên thiết bị.
• Sysfs: các thuộc tính chỉ đọc buffer_len, open_count và last_data được tạo bằng DEVICE_ATTR_RO để biểu diễn từng thuộc tính của thiết bị.

Các thành phần chính tìm thấy trong source gồm DEFINE_MUTEX(lab2_mutex), copy_from_user(), copy_to_user(), struct file_operations lab2_fops, register_chrdev(), proc_create(), device_create_file(), module_init() và module_exit().

4. KẾT QUẢ THỰC NGHIỆM

4.1. Phần A – Character Device Driver

Module được biên dịch bằng Makefile sử dụng Kernel headers tại ~/embedded_lab1/kernel/linux-5.15, ARCH=arm và CROSS_COMPILE=arm-linux-gnueabihf-. Module lab2_driver.ko được tích hợp vào Initramfs tại /lib/modules/5.15.0/.

Kết quả kiểm thử ghi nhận: module được nạp thành công; /dev/lab2 là thiết bị ký tự major 240, minor 0; dữ liệu “Hello from userspace, LAB-02!” (30 byte) được ghi và đọc lại chính xác. Các thông điệp Message_1, Message_2 và Message_3 cũng được kiểm tra theo cơ chế round-trip. File driver_test.txt kết thúc với trạng thái Test COMPLETED và COMMAND_EXIT_CODE=0.

4.2. Phần B – Procfs và Sysfs

Trước khi ghi dữ liệu, Procfs báo major 240, buffer length 0, open count 0 và device name lab2. Sau khi ghi dữ liệu kiểm thử, buffer length thay đổi thành 30 và open count tăng lên 1. Sysfs trả về buffer_len=30, open_count=1 và last_data=Embedded Linux Lab2 Test Data. Log procfs_test.txt ghi nhận kiểm thử hoàn tất với mã thoát 0.

4.3. Phần C – NAND Simulator và JFFS2

Phần này được thực hiện trên Ubuntu Host, không chạy bên trong QEMU. Module nandsim tạo phân vùng NAND giả lập /dev/mtd2 có dung lượng 32 MiB, eraseblock 16 KiB, page/minimum I/O 512 byte và OOB 16 byte. Các phân vùng mtd0 và mtd1 được nhận diện là BIOS/BIOS1; thao tác thử nghiệm được giới hạn ở mtd2.

Sau khi xóa vùng thử nghiệm, ảnh JFFS2 được tạo từ thư mục dữ liệu mẫu và ghi vào /dev/mtd2 bằng nandwrite. Filesystem được mount qua /dev/mtdblock2 tại /mnt/jffs2. Ba file hello.txt, info.txt và logs/boot.log được đọc thành công. Tiếp đó, runtime.txt được tạo, unmount/remount và đọc lại đúng nội dung, xác nhận dữ liệu tồn tại sau chu kỳ mount.

Lệnh nanddump đọc 512 byte đầu tiên; hexdump hiển thị magic bytes của JFFS2 và chuỗi “Hello from Embedded Linux LAB-02!”. Báo cáo log ghi nhận ECC failed=0, corrected=0, bad blocks=0. Có cảnh báo nandwrite về các block chỉ chứa 0xff; tuy nhiên thao tác mount, đọc file và kiểm tra persistence đều thành công, nên cảnh báo này không làm thất bại bài kiểm thử.

Sau khi kiểm thử, filesystem được unmount và module nandsim được gỡ. /proc/mtd trở về chỉ còn mtd0 và mtd1, cho thấy phân vùng NAND giả lập đã được dọn khỏi hệ thống.

4.4. Phần D – Tự động nạp driver khi khởi động

BusyBox Init đọc /etc/inittab. Dòng ::sysinit:/etc/init.d/rcS gọi script rcS để mount proc, sysfs, tmpfs và devtmpfs, thiết lập hostname, sau đó kiểm tra sự tồn tại của /lib/modules/5.15.0/lab2_driver.ko và gọi insmod. Khi nạp thành công, script tạo /dev/lab2 với major 240, minor 0 và đặt quyền 666. Dòng ::askfirst:-/bin/sh mở shell tương tác sau khi sysinit hoàn tất.

Boot log ghi nhận các thông báo lab2_driver loaded OK, /dev/lab2 ready (major=240, minor=0) và System initialization complete. Quá trình kết thúc với COMMAND_EXIT_CODE=0.

5. SO SÁNH CÁC HỆ THỐNG TẬP TIN NHÚNG

Bảng dưới đây tóm tắt đặc điểm điển hình. Khả năng ghi và wear leveling thực tế còn phụ thuộc thiết bị lưu trữ, driver MTD/FTL và cấu hình hệ thống.

Tiêu chí	ext4	JFFS2	UBIFS	SquashFS	cramfs
Có ghi	Có	Có	Có	Không (RO)	Không (RO)
Loại thiết bị phù hợp	Block device / eMMC / SSD	Raw NOR/NAND nhỏ	Raw NAND lớn	Firmware/rootfs RO	Firmware RO cũ/nhỏ
Wear leveling	Thường do FTL/thiết bị đảm nhiệm	Có cơ chế quản lý erase/garbage collection	UBI cung cấp wear leveling và quản lý bad block	Không tự quản lý vì chỉ đọc	Không tự quản lý vì chỉ đọc
Nén dữ liệu	Không phải đặc tính chính	Có	Có thể dùng compression	Có	Có
Mount/khởi động	Tương đối nhanh, tùy thiết bị và kích thước	Có thể chậm do quét flash khi mount	Thiết kế tốt cho NAND lớn; mount không cần quét toàn bộ như JFFS2	Nhanh, nội dung chỉ đọc	Thường nhẹ, nhưng định dạng cũ
Ưu điểm chính	Phổ biến, nhiều tính năng, journal	Đơn giản, phù hợp raw flash nhỏ	Khả năng mở rộng và quản lý NAND tốt	Nén tốt, phù hợp rootfs bất biến	Nhẹ, cấu trúc đơn giản
Hạn chế chính	Không truy cập raw NAND trực tiếp theo cách thông thường	Mount chậm khi dung lượng lớn	Cấu hình UBI/MTD phức tạp hơn	Không ghi trực tiếp lên filesystem	Ít linh hoạt, hạn chế hơn các lựa chọn mới

Đề xuất cho thiết bị IoT dùng NAND 32 MB và có OTA:

Với raw NAND 32 MB, JFFS2 là lựa chọn phù hợp cho phân vùng dữ liệu ghi/đọc tương đối nhỏ nhờ hỗ trợ raw flash, nén và quản lý các erase block. Nếu thiết kế cần root filesystem ổn định, có thể cân nhắc SquashFS cho rootfs chỉ đọc kết hợp một phân vùng dữ liệu ghi được bằng JFFS2. Với NAND dung lượng lớn hơn hoặc yêu cầu hiệu năng/khả năng mở rộng cao, UBIFS thường là lựa chọn tốt hơn.

OTA cần được thiết kế độc lập với filesystem: nên có cơ chế cập nhật an toàn như A/B image hoặc phân vùng dự phòng, xác minh chữ ký/hash trước khi kích hoạt, kiểm tra tính toàn vẹn sau ghi và có khả năng rollback khi khởi động bản mới thất bại. Trên thiết bị chỉ có 32 MB, cần tính toán kỹ dung lượng dành cho image dự phòng, dữ liệu và vùng metadata; không nên tuyên bố A/B khả thi nếu chưa kiểm tra ngân sách bộ nhớ thực tế.

6. CÂU HỎI PHÂN TÍCH

Câu 1. Vì sao không dùng memcpy() để sao chép trực tiếp giữa kernel buffer và user buffer?

Kernel space và user space là hai miền địa chỉ có quyền truy cập và quy tắc kiểm tra khác nhau. Con trỏ do tiến trình truyền vào driver là địa chỉ user space; kernel không được giả định rằng con trỏ đó hợp lệ hoặc có thể dereference an toàn. copy_from_user() và copy_to_user() là các hàm chuyên dụng để kiểm tra và xử lý việc truy cập bộ nhớ người dùng, đồng thời trả về số byte chưa sao chép nếu có lỗi. Dùng memcpy() trực tiếp có thể gây page fault không được xử lý đúng, truy cập địa chỉ không hợp lệ, làm lỗi kernel hoặc gây rủi ro bảo mật. Vì vậy driver dùng copy_from_user() khi nhận dữ liệu và copy_to_user() khi trả dữ liệu.

Câu 2. Vì sao DEFINE_MUTEX bảo vệ device_buffer? Điều gì xảy ra nếu hai tiến trình ghi đồng thời?

device_buffer là vùng dữ liệu dùng chung giữa các lời gọi read/write và có thể bị nhiều tiến trình truy cập đồng thời. Mutex bảo đảm chỉ một luồng thực thi tại một thời điểm đi vào vùng tới hạn. Nếu không khóa, hai tiến trình có thể xen kẽ thao tác ghi, ghi đè dữ liệu của nhau, làm nội dung buffer bị trộn hoặc trạng thái như độ dài buffer không nhất quán. Đây là race condition. Mutex giúp thao tác cập nhật buffer và metadata liên quan diễn ra tuần tự, có thể dự đoán.

Câu 3. So sánh Procfs và Sysfs. Vì sao không nên thêm file mới vào /proc?

Procfs (/proc) chủ yếu cung cấp thông tin và giao diện liên quan đến tiến trình, trạng thái và hoạt động của kernel. Sysfs (/sys) biểu diễn cấu trúc thiết bị, bus, class, driver và thuộc tính của các đối tượng kernel theo mô hình thiết bị. Tài liệu kernel khuyến nghị không dùng /proc như nơi chứa mọi giao diện driver mới vì làm /proc mất mục đích tập trung, khó chuẩn hóa và khó duy trì. Trong bài lab, /proc/lab2_info phù hợp để xem một bản tóm tắt trạng thái driver; các thuộc tính riêng của thiết bị như buffer_len, open_count và last_data phù hợp với Sysfs vì chúng mô tả thuộc tính của thiết bị lab2.

Câu 4. Vì sao JFFS2 làm việc theo erase block trong khi ext4 ghi theo sector/block? Liên hệ đặc tính NAND.

NAND flash không thể ghi đè tùy ý lên dữ liệu cũ. Trước khi lập trình lại, một erase block phải được xóa; thao tác xóa diễn ra theo đơn vị erase block, trong khi thao tác ghi diễn ra theo page. NAND cũng có giới hạn số chu kỳ xóa/ghi và có thể xuất hiện bad block. JFFS2 được thiết kế cho raw flash và dùng cấu trúc log-structured: ghi dữ liệu mới vào vùng trống, đánh dấu dữ liệu cũ không còn hiệu lực và thu gom/xóa block khi cần. ext4 hướng tới block device, nơi thiết bị thường cung cấp giao diện sector/block có thể ghi lại; việc xử lý đặc tính vật lý NAND thường được che bởi flash translation layer (FTL) trong eMMC/SSD hoặc lớp quản lý bên dưới. Vì vậy không thể áp dụng trực tiếp mô hình ghi của ext4 cho raw NAND.

Câu 5. Nếu insmod trong rcS thất bại, quá trình boot có tiếp tục không? Cần cải thiện thế nào?

Với script hiện tại, nếu insmod thất bại, nhánh else chỉ in thông báo lỗi. Script không gọi exit với mã lỗi và cũng không đặt chế độ dừng khi gặp lỗi, vì vậy rcS tiếp tục chạy đến cuối. Khi sysinit script kết thúc, BusyBox init có thể tiếp tục thực hiện askfirst và mở shell. Đây là hành vi hợp lý nếu driver là thành phần tùy chọn, không thiết yếu để hệ thống khởi động.

Có thể phân loại lỗi thành: (1) lỗi nghiêm trọng như không mount được filesystem thiết yếu, không có console hoặc không thể khởi tạo thành phần bắt buộc – cần dừng hoặc chuyển sang recovery; (2) lỗi không nghiêm trọng như module thử nghiệm không nạp được – ghi log/cảnh báo, bỏ qua chức năng phụ và tiếp tục boot. Nên kiểm tra mã trả về của từng lệnh mount/insmod, ghi lỗi vào log, kiểm tra sự tồn tại của node thiết bị sau khi nạp, và định nghĩa rõ chính sách fatal/non-fatal. Nếu driver bắt buộc, có thể thoát với mã khác 0 hoặc chuyển sang chế độ recovery; nếu tùy chọn, tiếp tục boot nhưng thông báo chức năng thiết bị không khả dụng.

7. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC

1. Phân biệt phân vùng MTD thử nghiệm với các phân vùng hệ thống: /proc/mtd hiển thị mtd0 (BIOS), mtd1 (BIOS1) và mtd2 (NAND simulator partition 0). Để tránh tác động lên phân vùng hệ thống, thao tác erase/write/mount chỉ được thực hiện trên /dev/mtd2. Sau kiểm thử, nandsim được gỡ và danh sách MTD trở lại trạng thái ban đầu.

2. Tích hợp module vào Initramfs: driver phải được biên dịch đúng theo Kernel mục tiêu ARM 5.15.0 và được đặt đúng đường dẫn /lib/modules/5.15.0/lab2_driver.ko. Makefile sử dụng ARCH=arm và CROSS_COMPILE=arm-linux-gnueabihf-. Sau đó Initramfs được đóng gói lại và kiểm tra boot log để xác nhận module tự nạp.

3. Xác minh dữ liệu JFFS2 sau khi ghi: chỉ nhìn thấy thông báo nandwrite là chưa đủ. Đã mount filesystem, đọc các file mẫu, tạo runtime.txt, unmount/remount và đọc lại để xác nhận tính tồn tại của dữ liệu. Đồng thời dùng nanddump/hexdump để quan sát dữ liệu thô trên NAND.

8. KẾT LUẬN

Bài thực hành đã triển khai được Character Device Driver trên Linux ARM, kiểm tra luồng đọc/ghi an toàn giữa user space và kernel space, bảo vệ buffer bằng mutex, xuất thông tin qua Procfs/Sysfs, tạo và kiểm tra JFFS2 trên NAND Simulator, đồng thời cấu hình BusyBox Init tự động nạp driver khi khởi động. Các log kiểm thử A, B, C và D đều ghi nhận kết quả hoàn tất với mã thoát 0. Qua bài lab, người thực hiện hiểu rõ hơn sự khác biệt giữa block device và raw flash, vai trò của MTD/JFFS2, cũng như quy trình tích hợp driver vào root filesystem nhúng.

PHỤ LỤC – CÁC FILE MINH CHỨNG

logs/boot_log.txt – log khởi động và tự động nạp module.
logs/driver_test.txt – log kiểm thử Character Device.
logs/procfs_test.txt – log kiểm thử Procfs và Sysfs.
logs/mtd_jffs2.txt – log NAND Simulator, tạo/ghi/mount JFFS2 và kiểm tra dữ liệu.

> Các file log gốc cần được giữ nguyên trong thư mục nộp bài để giảng viên đối chiếu. Báo cáo này là bản tổng hợp; nếu yêu cầu của giảng viên có thêm ảnh chụp màn hình hoặc mẫu trình bày riêng, cần bổ sung trước khi nộp chính thức.